

Методический материал для студентов

Введение в PyTorch: подключение, тензоры, готовые модели и AI-модуль в проекте

Два учебных занятия по 80 минут

Цель материала

Этот документ можно использовать как готовую методичку для проведения двух пар. Внутри есть теория, полные блоки кода для Google Colab, объяснение по строкам, вопросы студентам, практические задания и домашняя работа.

Пара	Главный фокус	Результат
1 пара	PyTorch, Tensor, device, первая модель через nn.Sequential	Студент понимает, как подключить PyTorch, создать tensor и пропустить данные через модель
2 пара	Готовые модели, Hugging Face pipeline, AI-модуль и архитектура проекта	Студент понимает, как использовать готовую модель и где она живет в реальном проекте

Как использовать эту методичку

- Материал рассчитан на Google Colab. Все кодовые блоки можно копировать и запускать по порядку.
- Если у студентов слабая база, не ускоряйтесь: лучше меньше тем, но чтобы они поняли Tensor, model(X), output и argmax.
- На этих двух парах не нужно обучать большую модель с нуля. Главная цель - понять PyTorch как инструмент для AI-проектов.
- Готовые модели показаны через Hugging Face Transformers, потому что это самый понятный вход в реальное применение AI в проектах.

Предварительные знания

- Python: переменные, списки, функции, импорт библиотек.
- Базовый ML: признаки X, целевая переменная y, train/test, fit, predict, accuracy.
- Понимание, что модель получает данные и возвращает предсказание.
- Минимальное понимание нейрона: входы умножаются на веса, добавляется bias, затем применяется активация.

Большая идея: чем PyTorch отличается от sklearn

В sklearn мы часто работаем с моделью как с готовым объектом:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Это удобно, но метод `fit` скрывает большую часть процесса обучения. PyTorch дает больше контроля: мы явно видим данные, модель, ошибку, градиенты и обновление весов.

```
# Общая логика PyTorch
# 1. Подготовить данные как Tensor
# 2. Создать модель
# 3. Получить output = model(X)
# 4. Посчитать loss
# 5. Сделать loss.backward()
# 6. Сделать optimizer.step()
```

Критерий	sklearn	PyTorch
Основной фокус	Классическое машинное обучение	Нейросети и deep learning
Обучение	<code>model.fit()</code>	training loop: forward, loss, backward, optimizer.step()
Предсказание	<code>model.predict()</code>	<code>model(X)</code> , затем обработка output
GPU	Обычно не используется напрямую	Поддерживается через cuda
Гибкость	Меньше контроля, проще старт	Больше контроля, больше кода
Готовые модели	Ограниченно	Много: vision, text, audio, transformers

Пара 1. PyTorch с нуля: подключение, Tensor, device, первая модель

Время	Блок	Что делаем
0-10 мин	Мостик от sklearn	Повторяем fit/predict и объясняем, зачем нужен PyTorch
10-20 мин	Что такое PyTorch	Разбираем torch, Tensor, nn, optim, Dataset, DataLoader
20-35 мин	Подключение	Проверяем версию, GPU и device
35-50 мин	Tensor	Создаем одномерные и

		двумерные тензоры
50-60 мин	dtype float32	Объясняем типы данных и почему int плохо для нейросетей
60-70 мин	Операции	Сложение, умножение, mean, sum
70-80 мин	Первая модель	nn.Sequential, Linear, ReLU, output, argmax

1.1. Вступление: зачем нам PyTorch после sklearn

Сначала важно не бросать студентов сразу в синтаксис. Нужно объяснить, что PyTorch - это следующий уровень после sklearn. В sklearn мы быстро обучали готовые модели, а в PyTorch можем контролировать внутреннюю механику нейросети.

Фраза для объяснения

sklearn удобен, когда нам нужна готовая классическая модель. PyTorch нужен, когда мы хотим строить нейросети, работать с GPU, подключать готовые AI-модели и внедрять их в проекты.

- В sklearn мы говорили модели: обучись.
- В PyTorch мы сами видим путь данных через модель.
- В sklearn многое спрятано в fit().
- В PyTorch мы можем вручную управлять forward pass, loss, backward pass и optimizer.

1.2. Экосистема PyTorch

PyTorch - это не одна функция. Это экосистема из нескольких важных частей.

Компонент	Назначение	Пример
torch	Основная библиотека: тензоры, математика, GPU	torch.tensor(), torch.cuda.is_available()
torch.Tensor	Главный формат данных в PyTorch	torch.tensor([1, 2, 3])
torch.nn	Слои и функции для построения нейросетей	nn.Linear, nn.ReLU, nn.Sequential
torch.optim	Оптимизаторы для обновления весов	SGD, Adam, AdamW
torch.utils.data	Работа с Dataset и DataLoader	DataLoader(dataset, batch_size=32)
torchvision	Инструменты и модели для изображений	ResNet, MobileNet, transforms
torchaudio	Инструменты для звука	audio datasets, transforms
transformers	Готовые NLP/LLM-модели поверх PyTorch	pipeline("sentiment-analysis")

1.3. Подключение PyTorch в Google Colab

В Google Colab PyTorch обычно уже установлен. Начинаем с проверки версии.

```
# Блок 1. Проверка PyTorch
import torch

print("Версия PyTorch:", torch.__version__)
```

Объяснение

import torch подключает библиотеку PyTorch. torch.__version__ показывает, какая версия установлена в окружении.

1.4. Проверка GPU

GPU нужен для ускорения обучения больших нейросетей. На первом уроке мы не будем обучать большую модель, но студенты должны понимать, почему device важен.

```
# Блок 2. Проверка GPU
import torch

gpu_available = torch.cuda.is_available()

print("GPU доступен:", gpu_available)
```

- Если вывод True - доступна видеокарта.
- Если вывод False - код будет работать на CPU.
- Для маленьких примеров CPU достаточно.
- Для больших нейросетей GPU сильно ускоряет обучение.

1.5. Выбор устройства: CPU или CUDA

```
# Блок 3. Выбор устройства
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Используем устройство:", device)
```

В проектах эту конструкцию используют почти всегда. Она позволяет одному и тому же коду работать и на обычном ноутбуке, и на сервере с GPU.

```
# Позже модель и данные можно отправить на device так:
# model = model.to(device)
# X = X.to(device)
# y = y.to(device)
```

Важная ошибка новичков

Если модель находится на GPU, а данные на CPU, PyTorch выдаст ошибку. Модель и данные должны быть на одном устройстве.

1.6. Tensor - основной объект PyTorch

Tensor - это массив чисел, с которым работает PyTorch. Он похож на NumPy array, но умеет работать на GPU и участвовать в вычислении градиентов.

```
# Блок 4. Создание простого tensor
import torch
```

```
x = torch.tensor([1, 2, 3])

print("Tensor:", x)
print("Тип объекта:", type(x))
print("Форма tensor:", x.shape)
print("Тип данных внутри tensor:", x.dtype)
```

Команда	Что показывает
type(x)	Тип объекта Python
x.shape	Форму tensor
x.dtype	Тип чисел внутри tensor

1.7. Двумерный Tensor как таблица признаков

```
# Блок 5. Двумерный tensor
import torch

X = torch.tensor([
    [1, 2, 3],
    [4, 5, 6]
])

print("Tensor:")
print(X)

print("Форма:", X.shape)
print("Количество измерений:", X.ndim)
print("Тип данных:", X.dtype)
```

Форма [2, 3] означает: 2 строки и 3 столбца. В машинном обучении это можно читать так: 2 объекта и 3 признака.

```
# Пример: 2 клиента, 3 признака
# признаки: age, salary, orders_count
customers = torch.tensor([
    [18, 150000, 3],
    [25, 300000, 10]
])

print(customers)
print(customers.shape)
```

1.8. Почему для нейросетей нужен float32

Если создать tensor из целых чисел, PyTorch обычно создаст int64. Для нейросетей чаще нужен float32, потому что веса, ошибки и градиенты являются дробными числами.

```
# Блок 6. Tensor с целыми числами
import torch

x = torch.tensor([1, 2, 3])

print(x)
print(x.dtype)
```

```
# Блок 7. Правильный tensor для нейросети
```

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)

print(x)
print(x.dtype)
```

Главная мысль

Для входов в нейросеть обычно используем `dtype=torch.float32`. Это стандартный тип для большинства операций deep learning.

1.9. Операции с Tensor

```
# Блок 8. Математика с tensor
import torch

a = torch.tensor([1, 2, 3], dtype=torch.float32)
b = torch.tensor([10, 20, 30], dtype=torch.float32)

print("a:", a)
print("b:", b)

print("Сложение:", a + b)
print("Умножение:", a * b)
print("Сумма a:", a.sum())
print("Среднее a:", a.mean())
print("Максимум a:", a.max())
print("Минимум a:", a.min())
```

Внутри нейросети постоянно выполняются операции умножения, сложения, суммирования и усреднения. Нейрон можно представить как формулу:

```
# Идея одного нейрона:
# output = x1*w1 + x2*w2 + x3*w3 + b
```

1.10. Первая модель в PyTorch без обучения

На этом этапе не обучаем модель. Цель - увидеть, как данные проходят через модель и как получается output.

```
# Блок 9. Простая нейросеть
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
)

print(model)
```

Часть модели	Смысл
<code>nn.Sequential</code>	Собирает модель как цепочку слоев
<code>nn.Linear(4, 8)</code>	Принимает 4 входных числа и выдает 8 чисел

nn.ReLU()	Функция активации, добавляет нелинейность
nn.Linear(8, 3)	Принимает 8 чисел и выдает 3 числа

1.11. Пропускаем данные через модель

```
# Блок 10. Даем данные модели
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
)

X = torch.tensor([
    [5.1, 3.5, 1.4, 0.2]
], dtype=torch.float32)

output = model(X)

print("Входные данные:")
print(X)

print("Выход модели:")
print(output)

print("Форма выхода:", output.shape)
```

Мы дали модели один объект с 4 признаками. Модель вернула 3 числа. Эти числа называются logits - сырые оценки классов. Это еще не вероятности.

1.12. Получаем предсказанный класс через argmax

```
# Блок 11. Получаем предсказанный класс
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
)

X = torch.tensor([
    [5.1, 3.5, 1.4, 0.2]
], dtype=torch.float32)

output = model(X)

predicted_class = torch.argmax(output, dim=1)

print("Output:", output)
print("Predicted class:", predicted_class.item())
```

Важно

Пока модель не обучена, ее предсказание случайное. Мы сейчас изучаем форму кода и

прохождение данных, а не качество модели.

1.13. Мини-практика в конце первой пары

1. Создать tensor размером 3x4 с любыми числами типа float32.
2. Вывести shape, dtype, mean, sum.
3. Создать модель: Linear(4, 10), ReLU, Linear(10, 3).
4. Пропустить tensor через модель.
5. Получить predicted_class через torch.argmax(output, dim=1).
6. Письменно объяснить, почему output имеет форму [3, 3].

```
# Шаблон для мини-практики
import torch
import torch.nn as nn

X = torch.tensor([
    [1.0, 2.0, 3.0, 4.0],
    [5.0, 6.0, 7.0, 8.0],
    [9.0, 10.0, 11.0, 12.0]
], dtype=torch.float32)

model = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

output = model(X)
predicted_classes = torch.argmax(output, dim=1)

print("X shape:", X.shape)
print("Output shape:", output.shape)
print("Output:", output)
print("Predicted classes:", predicted_classes)
```

Итог первой пары

- PyTorch подключается через import torch.
- Tensor - основной формат данных в PyTorch.
- dtype=torch.float32 чаще всего нужен для нейросетей.
- device нужен для выбора CPU или GPU.
- Модель можно создать через nn.Sequential.
- model(X) пропускает данные через модель.
- output - это сырые оценки классов.
- argmax выбирает индекс самого большого значения.

Пара 2. Готовые модели и использование AI-модели в проекте

Время	Блок	Что делаем
0-10 мин	Повторение	Tensor, shape, dtype, nn.Linear, model(X), argmax
10-25 мин	Готовые модели	Что такое pretrained model и где применяется
25-45 мин	Sentiment analysis	Подключаем Hugging Face

		pipeline и анализируем текст
45-55 мин	Несколько отзывов	Пропускаем список отзывов через модель
55-65 мин	Функция для проекта	Пишем analyze_review(text)
65-75 мин	Архитектура	Frontend -> Backend API -> AI model -> Database
75-80 мин	Закрепление	Мини-практика и ответы на вопросы

2.1. Быстрое повторение первой пары

```
# Повторение: модель, device, output, argmax
import torch
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
).to(device)

X = torch.tensor([
    [5.1, 3.5, 1.4, 0.2]
], dtype=torch.float32).to(device)

output = model(X)
predicted_class = torch.argmax(output, dim=1)

print("Device:", device)
print("Output:", output)
print("Predicted class:", predicted_class.item())
```

Вопросы для повторения

- Что такое Tensor?
- Что показывает shape?
- Почему часто нужен float32?
- Что делает nn.Linear?
- Что делает ReLU?
- Что делает model(X)?
- Что делает argmax?
- Что такое device?

2.2. Что такое готовая модель

В реальной разработке не всегда обучают модель с нуля. Часто берут уже обученную модель и используют ее в своем проекте. Такая модель называется pretrained model, то есть предобученная модель.

```
# Идея готовой модели:
# 1. Берем модель, которую уже обучили на большом датасете
# 2. Даем ей свои данные
# 3. Получаем результат
```

Тип модели	Примеры задач
Текст	Анализ отзывов, классификация сообщений, перевод, суммаризация, чат-боты
Изображения	Классификация фото, поиск объектов, медицинские снимки, контроль качества
Аудио	Speech-to-text, классификация звуков, анализ звонков
Табличные данные	Риски, прогноз покупок, скоринг, рекомендации
Мультимодальные модели	Текст + изображение, товар + описание + отзывы

Честное объяснение студентам

Для табличных данных часто лучше подходят Random Forest, XGBoost, CatBoost или Logistic Regression. PyTorch особенно полезен, когда нужны нейросети, большие данные, кастомная архитектура или объединение разных типов данных.

2.3. Готовая модель для анализа тональности текста

Для практики используем библиотеку Hugging Face Transformers. Она позволяет очень быстро подключать готовые модели для текста. Часто такие модели работают поверх PyTorch.

```
# Блок 12. Установка библиотеки Transformers
!pip install transformers
```

Для Colab

Если библиотека уже установлена, Colab напишет Requirement already satisfied. Это нормально.

2.4. Первый запуск sentiment-analysis

```
# Блок 13. Первая готовая модель для текста
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

text = "I really like this course!"

result = classifier(text)

print(result)
```

Строка	Объяснение
from transformers import pipeline	Импортируем удобную функцию для готовых моделей

pipeline("sentiment-analysis")	Создаем модель для анализа тональности текста
classifier(text)	Передаем текст в модель и получаем результат
label	Предсказанный класс: POSITIVE или NEGATIVE
score	Уверенность модели

2.5. Анализ нескольких отзывов

```
# Блок 14. Анализ списка отзывов
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

reviews = [
    "I love this product!",
    "This app is terrible.",
    "The lesson was useful and clear.",
    "I don't like the design.",
    "The website works very slowly.",
    "This course helped me understand AI.",
    "The service is bad.",
    "The interface is simple and comfortable.",
    "I am not happy with the quality.",
    "Everything works perfectly."
]

results = classifier(reviews)

for review, result in zip(reviews, results):
    print("Review:", review)
    print("Label:", result["label"])
    print("Score:", round(result["score"], 4))
    print("-" * 50)
```

Что объяснить

classifier(reviews) принимает список текстов и возвращает список результатов. zip(reviews, results) соединяет каждый отзыв с его предсказанием.

2.6. Заворачиваем модель в функцию для проекта

В реальном проекте модель лучше завернуть в функцию. Тогда backend сможет просто вызвать функцию и получить результат.

```
# Блок 15. AI-функция для проекта
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

def analyze_review(text):
    # Функция принимает текст отзыва
    # и возвращает результат анализа тональности.
    result = classifier(text)[0]

    return {
```

```

        "text": text,
        "label": result["label"],
        "score": round(result["score"], 4)
    }

```

```
review = "This application is very useful for students."
```

```
prediction = analyze_review(review)
```

```
print(prediction)
```

Часть кода	Смысл
classifier = pipeline(...)	Загружаем модель один раз
def analyze_review(text)	Создаем функцию для анализа текста
classifier(text)[0]	Получаем первый результат модели
return {...}	Возвращаем удобный словарь для проекта

2.7. Анализ отзывов через функцию

Блок 16. Использование функции для нескольких отзывов

```

reviews = [
    "The course is very helpful.",
    "The application is slow.",
    "I like the interface.",
    "The quality is bad.",
    "Everything is excellent."
]

for review in reviews:
    prediction = analyze_review(review)

    print("Text:", prediction["text"])
    print("Label:", prediction["label"])
    print("Score:", prediction["score"])
    print("-" * 50)

```

2.8. Как AI-модель используется в проекте

AI-модель редко существует сама по себе. Обычно она является частью backend-сервиса или отдельного AI-сервиса.

```

# Общая архитектура:
# Frontend
#   ↓
# Backend API
#   ↓
# AI model / AI service
#   ↓
# Prediction
#   ↓
# Database
#   ↓
# Frontend / Admin panel

```

Этап	Что происходит
------	----------------

Frontend	Пользователь вводит отзыв, сообщение или данные
Backend API	Принимает запрос от frontend
AI-модель	Делает предсказание: например, POSITIVE/NEGATIVE
Database	Сохраняет исходный текст, label и score
Admin panel	Показывает статистику и аналитику

2.9. Пример структуры проекта

```
ai-review-project/
├── app.py
├── ai/
│   ├── model.py
│   └── predictor.py
├── api/
│   └── routes.py
├── data/
│   └── reviews.csv
└── requirements.txt
```

Файл	Назначение
app.py	Главный файл запуска приложения
ai/model.py	Загрузка готовой модели
ai/predictor.py	Функция analyze_review или predict
api/routes.py	API-маршруты backend
data/reviews.csv	Данные или тестовые отзывы
requirements.txt	Список библиотек проекта

2.10. Мини-версия AI-модуля по файлам

Это не обязательно запускать на уроке, но полезно показать как архитектуру.

Файл ai/model.py

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
```

Файл ai/predictor.py

```
from ai.model import classifier

def analyze_review(text):
    result = classifier(text)[0]

    return {
        "text": text,
        "label": result["label"],
```

```
        "score": round(result["score"], 4)
    }
```

Файл app.py

```
from ai.predictor import analyze_review

review = "The platform is useful and simple."

result = analyze_review(review)

print(result)
```

2.11. Практическая работа в конце второй пары

7. Установить transformers.
8. Создать pipeline("sentiment-analysis").
9. Создать список из 10 отзывов на английском языке.
10. Получить label и score для каждого отзыва.
11. Создать функцию analyze_review(text).
12. Объяснить, где такую функцию можно использовать в проекте.

```
# Шаблон практической работы
!pip install transformers

from transformers import pipeline

classifier = pipeline("sentiment-analysis")

def analyze_review(text):
    result = classifier(text)[0]
    return {
        "text": text,
        "label": result["label"],
        "score": round(result["score"], 4)
    }

reviews = [
    "This platform is very useful.",
    "The design is bad.",
    "I like the lesson.",
    "The app is too slow.",
    "Everything works well.",
    "Support is not helpful.",
    "The course is clear.",
    "The interface is confusing.",
    "I am happy with the result.",
    "The quality is terrible."
]

for review in reviews:
    result = analyze_review(review)
    print(result)
```

Итог второй пары

- Готовая модель - это модель, которую уже обучили до нас.
- Hugging Face pipeline позволяет быстро использовать такие модели.
- Sentiment-analysis определяет настроение текста.

- AI-логику лучше заворачивать в функцию.
- В реальном проекте AI-модель вызывается backend-ом или отдельным AI-сервисом.
- Результат модели можно сохранить в базу данных или показать на frontend/admin panel.

Типичные ошибки студентов и как их объяснить

Ошибка	Почему возникает	Как исправить
ModuleNotFoundError: transformers	Библиотека не установлена	Запустить <code>!pip install transformers</code>
RuntimeError про cuda/cpu	Модель и данные на разных устройствах	Перенести все на один device: <code>model.to(device), X.to(device)</code>
Expected Float but got Long	Данные созданы как int64	Добавить <code>dtype=torch.float32</code>
Непонятный output модели	Модель возвращает logits, а не готовый текст	Для класса использовать <code>torch.argmax(output, dim=1)</code>
Предсказания случайные	Модель nn.Sequential еще не обучена	Объяснить, что сейчас изучается структура, не качество
Colab долго скачивает модель	Transformers скачивает pretrained model	Подождать загрузку, затем модель будет работать быстрее

Полный Colab-код для студентов

Этот раздел можно дать студентам как итоговый notebook. Код можно запускать блоками сверху вниз.

Блок 1. Проверка PyTorch и device

```
import torch

print("Версия PyTorch:", torch.__version__)
print("GPU доступен:", torch.cuda.is_available())

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Используем устройство:", device)
```

Блок 2. Tensor

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)

print("Tensor:", x)
print("Shape:", x.shape)
print("Dtype:", x.dtype)
```

Блок 3. Двумерный Tensor

```
import torch

X = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
    [7.0, 8.0, 9.0]
])

print("X:")
print(X)

print("Shape:", X.shape)
print("Количество измерений:", X.ndim)
print("Среднее:", X.mean())
print("Сумма:", X.sum())
```

Блок 4. Простая модель PyTorch

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 6),
    nn.ReLU(),
    nn.Linear(6, 2)
)

print(model)
```

Блок 5. Пропускаем данные через модель

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 6),
    nn.ReLU(),
    nn.Linear(6, 2)
)

X = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
    [7.0, 8.0, 9.0]
])

output = model(X)

print("Input:")
print(X)

print("Output:")
print(output)

print("Output shape:", output.shape)
```


Блок 6. Получаем классы через argmax

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 6),
    nn.ReLU(),
    nn.Linear(6, 2)
)

X = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
    [7.0, 8.0, 9.0]
])

output = model(X)

predicted_classes = torch.argmax(output, dim=1)

print("Output:")
print(output)

print("Predicted classes:")
print(predicted_classes)
```

Блок 7. Установка Transformers

```
!pip install transformers
```

Блок 8. Готовая модель sentiment-analysis

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

text = "I really like this course!"

result = classifier(text)

print(result)
```

Блок 9. Анализ списка отзывов

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

reviews = [
    "I love this product!",
    "This app is terrible.",
    "The lesson was useful and clear.",
    "I don't like the design.",
    "The website works very slowly.",
    "This course helped me understand AI.",
    "The service is bad.",
    "The interface is simple and comfortable.",
    "I am not happy with the quality.",
    "Everything works perfectly."
```

```

]

results = classifier(reviews)

for review, result in zip(reviews, results):
    print("Review:", review)
    print("Label:", result["label"])
    print("Score:", round(result["score"], 4))
    print("-" * 50)

```

Блок 10. Функция для проекта

```

from transformers import pipeline

classifier = pipeline("sentiment-analysis")

def analyze_review(text):
    result = classifier(text)[0]

    return {
        "text": text,
        "label": result["label"],
        "score": round(result["score"], 4)
    }

review = "This application is very useful for students."

prediction = analyze_review(review)

print(prediction)

```

Блок 11. Анализ отзывов через функцию

```

reviews = [
    "The course is very helpful.",
    "The application is slow.",
    "I like the interface.",
    "The quality is bad.",
    "Everything is excellent."
]

for review in reviews:
    prediction = analyze_review(review)

    print("Text:", prediction["text"])
    print("Label:", prediction["label"])
    print("Score:", prediction["score"])
    print("-" * 50)

```

Задания для студентов на уроке

Задание 1. Tensor

- Создайте tensor размером 3x4.
- Все числа должны быть float32.
- Выведите shape, dtype, mean, sum.

- Объясните, что означает shape.

Задание 2. Простая модель

- Создайте модель: Linear(4, 8), ReLU, Linear(8, 3).
- Создайте 3 объекта, у каждого 4 признака.
- Пропустите данные через модель.
- Получите predicted_classes через argmax.

Задание 3. Готовая NLP-модель

- Установите transformers.
- Создайте pipeline("sentiment-analysis").
- Создайте 10 отзывов на английском языке.
- Выведите для каждого отзыва label и score.

Задание 4. AI-функция

- Создайте функцию analyze_review(text).
- Функция должна возвращать словарь с ключами text, label, score.
- Проверьте функцию на 5 новых отзывах.

Домашнее задание

Формат сдачи

- Сдать Google Colab notebook или .ipynb файл.
- Код должен запускаться сверху вниз без ручных исправлений.
- После каждого крупного блока должен быть короткий текстовый вывод или комментарий.

Часть 1. PyTorch basics

13. Подключить torch.
14. Вывести версию PyTorch.
15. Проверить GPU.
16. Создать device.
17. Создать tensor размером 3x4 с dtype=torch.float32.
18. Вывести shape, dtype, mean, sum.

Часть 2. Простая модель

19. Создать модель: Linear(4, 8), ReLU, Linear(8, 3).
20. Создать 3 объекта с 4 признаками.
21. Пропустить данные через модель.
22. Вывести output и output.shape.
23. Получить predictions через torch.argmax(output, dim=1).
24. Письменно объяснить, почему модель выдает 3 числа на каждый объект.

```
# Шаблон для домашнего задания, часть 2
import torch
import torch.nn as nn
```

```
model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
)
```

```

X = torch.tensor([
    [5.1, 3.5, 1.4, 0.2],
    [6.2, 3.4, 5.4, 2.3],
    [5.9, 3.0, 4.2, 1.5]
], dtype=torch.float32)

output = model(X)
predictions = torch.argmax(output, dim=1)

print("Output:")
print(output)

print("Predictions:")
print(predictions)

```

Часть 3. Готовая модель для текста

25. Установить transformers.
26. Создать classifier = pipeline("sentiment-analysis").
27. Создать список из 10 своих отзывов на английском языке.
28. Для каждого отзыва вывести text, label, score.

Часть 4. Функция для проекта

29. Создать функцию analyze_review(text).
30. Функция должна возвращать словарь с text, label, score.
31. Проверить функцию на 5 новых текстах.

Часть 5. Теория

32. Что такое PyTorch?
33. Что такое Tensor?
34. Чем Tensor отличается от NumPy array?
35. Зачем нужен torch.nn?
36. Что делает nn.Linear?
37. Что делает ReLU?
38. Что такое pretrained model?
39. Что такое inference?
40. Где можно использовать sentiment-analysis в реальном проекте?
41. Как backend может работать с AI-моделью?

Критерии оценки домашнего задания

Критерий	Баллы
Подключил PyTorch и проверил GPU	2
Создал device	1
Создал Tensor и вывел shape/dtype/mean/sum	3
Создал модель через nn.Sequential	3
Пропустил данные через модель	2
Получил классы через argmax	2

Использовал готовую модель sentiment-analysis	3
Обработал 10 отзывов	2
Написал функцию analyze_review	2
Ответил на теоретические вопросы	4
Итого	24

Глоссарий

Термин	Объяснение
PyTorch	Библиотека Python для нейросетей и deep learning
Tensor	Основной формат данных в PyTorch
GPU	Видеокарта, которая ускоряет обучение нейросетей
CUDA	Технология для вычислений на GPU NVIDIA
device	Устройство, на котором находятся модель и данные: cpu или cuda
nn.Linear	Полносвязный слой нейросети
ReLU	Функция активации, которая заменяет отрицательные значения на 0
logits	Сырые выходы модели до превращения в вероятности
argmax	Операция выбора индекса самого большого значения
pretrained model	Модель, заранее обученная на большом датасете
inference	Использование обученной модели для получения предсказания
pipeline	Удобная обертка Hugging Face для быстрого использования готовых моделей
AI-модуль	Отдельная часть проекта, в которой находится модель и функция predict/analyze

Что делать на следующем уроке

После этих двух пар логично переходить к обучению собственной модели в PyTorch. Следующий урок можно построить вокруг полного training loop.

- Dataset и DataLoader.

- Разделение train/test.
- Loss function.
- Optimizer.
- Training loop.
- Accuracy.
- Сохранение модели.
- Загрузка модели.
- Inference через свою обученную модель.

```
# Будущая схема обучения модели в PyTorch:  
# for epoch in range(num_epochs):  
#     for X_batch, y_batch in dataloader:  
#         output = model(X_batch)  
#         loss = criterion(output, y_batch)  
#         optimizer.zero_grad()  
#         loss.backward()  
#         optimizer.step()
```